

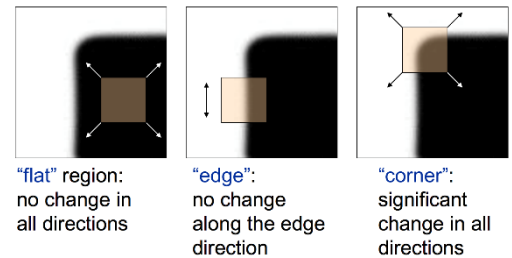
Video Stabilization

Parameter Exploration and Failure Cases

Harris Corner Detector

The Harris Corner Detector is a method to find corners in an image. Corners are points where the intensity changes strongly in all directions.

The idea is to take a small window around a pixel and measure how much the image intensity changes when the window is shifted. If the change is large in every direction, the point is likely a corner.



Parameter	Low Value	High Value
maxCorners	Too few features	Dense features, more computation
qualityLevel	Many weak features (unstable)	Few strong features (stable)
minDistance	Dense clusters	Sparse distribution
blockSize	Sensitive/noisy	Strong but fewer corners
apply_gaussian_blur	More details but more noise	Cleaner, fewer corners
gaussian_ksize	Light smoothing	Heavy smoothing, coarse features

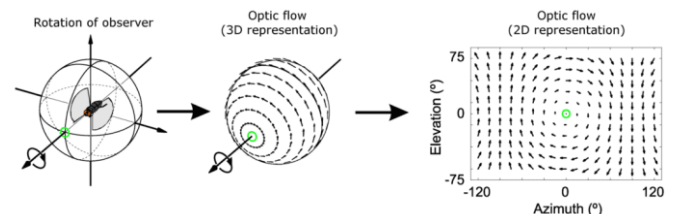
Visual effect overall:

Low thresholds and small blockSize produce many scattered features that jitter or disappear; high thresholds, larger neighborhoods and moderate blur produce fewer but much more stable tracks and smoother stabilization.

Lucas-Kanade Optical Flow

Optical Flow

Optical flow quantifies the motion of objects between consecutive frames captured by a camera. These algorithms attempt to capture the apparent motion of brightness patterns in the image.



Lucas-Kanade Method

The Lucas–Kanade method estimates how points move between two video frames.

It is based on two key assumptions:

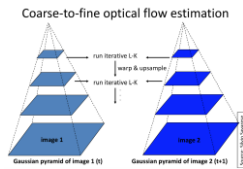
1. Brightness of a pixel stays the same between frames: $I(x, y, t) = I(x + dx, y + dy, t + dt)$
2. Neighboring pixels move in the same way. (all pixels inside this small block belong to the same object surface, so when the object moves, the whole block moves together):

$$f_x u + f_y v + f_t = 0 \quad f_x = \frac{\partial f}{\partial x}; f_y = \frac{\partial f}{\partial y} \quad u = \frac{dx}{dt}; v = \frac{dy}{dt}$$

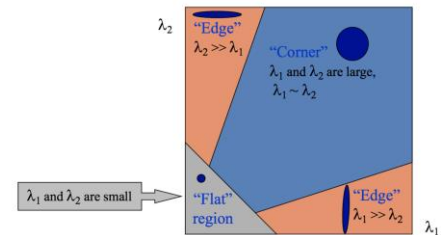
Because a single pixel does not give enough information to determine motion, Lucas–Kanade takes a small window (e.g., 3×3 or larger) around the pixel. All pixels in this window are assumed to have the same motion.

Using image gradients in x , y , and time, this window creates multiple equations that can be solved with least squares, producing the motion vector: (u, v) = the movement of the pixel

Corners are the best points to track because they have strong gradients in both directions (edges and flat areas don't work well).



Since Lucas–Kanade only handles small motions, it uses image pyramids to detect large motions by gradually refining motion from coarse to fine scales.



Parameter	Low Value Effect	High Value Effect
Window Size	Accurate for small motion, noise-sensitive	Robust to large motion, less precise
Pyramid Levels	Fast, only small displacements	Robust for large motion, higher compute
Max Iterations	Fast but less accurate	More accurate but slower
Epsilon	Precise convergence	Faster but less accurate

Visual effect overall:

Smaller windows, few pyramid levels, few iterations and large epsilon give fast but fragile tracking (many features die or jitter). Larger windows and more pyramid levels plus tighter convergence criteria yield stable, long-lived tracks and therefore smoother stabilized videos, at the cost of speed.

Failure Cases

- **Low-texture surfaces** (e.g., plain walls, sky):
 - Few or no detectable corners → KLT has nothing reliable to track.
 - Leads to drifting, disappearing, or noisy tracks.
- **Motion blur / fast camera movement:**
 - Feature appearance changes significantly; LK's small-motion and brightness-constancy assumptions are violated.
 - Many tracks fail or jump between frames.
- **Parallax or moving foreground objects:**
 - Different scene depths move inconsistently → cannot be represented by a single affine transform.
 - Causes wobbling or distorted stabilization.
- **Strong lighting changes:**
 - Harris responses weaken; LK mismatches features.
 - Results in unstable and inconsistent tracking.

Connecting Observations to the Implementation

The behavior seen during experimentation directly follows from how the detector and tracker are implemented in the code. The Harris detector relies on `cv2.goodFeaturesToTrack` with `useHarrisDetector=True`, and all GUI parameters (such as `qualityLevel`, `maxCorners`, `minDistance`, and `blockSize`) are passed straight into this function. This explains why lowering thresholds produced many weak, unstable corners and why increasing them resulted in fewer but stronger, more reliable points.

Likewise, the Lucas–Kanade tracker uses `cv2.calcOpticalFlowPyrLK`, with the window size, pyramid levels, and stopping criteria taken directly from the GUI settings and fed into `self.lk_params`. Because of this, small windows and shallow pyramids led to frequent tracking failures under fast motion, while larger settings produced more stable but more computationally expensive tracks. The resulting

point correspondences are then given to the stabilizer, so any instability in detection or tracking appears visibly as jitter or frame wobble in the stabilization output.

Implementing Video Stabilization

```

65     def stabilize(self, prev_pts, curr_pts):
66         if prev_pts is None or curr_pts is None or len(prev_pts) < 3:
67             M = np.float32([[1, 0, 0],
68                             [0, 1, 0]])
69             raw_delta = (0, 0)
70             smooth_delta = (0, 0)
71             return M, raw_delta, smooth_delta
72
73         prev_pts = prev_pts.reshape(-1, 1, 2).astype(np.float32)
74         curr_pts = curr_pts.reshape(-1, 1, 2).astype(np.float32)
75
76         T, inliers = cv2.estimateAffinePartial2D(
77             prev_pts, curr_pts,
78             method=cv2.RANSAC,
79             ransacReprojThreshold=3.0
80         )
81
82         if T is None:
83             T = np.float32([[1, 0, 0],
84                             [0, 1, 0]])
85
86         dx = T[0, 2]
87         dy = T[1, 2]
88         da = np.arctan2(T[1, 0], T[0, 0])
89
90         raw_delta = (dx, dy)
91
92         self.cumulative_x += dx
93         self.cumulative_y += dy
94         self.cumulative_a += da
95
96         curr_traj = np.array([self.cumulative_x,
97                               self.cumulative_y,
98                               self.cumulative_a])
99         self.trajectory.append(curr_traj)
100
101         radius = self.smoothing_radius
102         idx = len(self.trajectory) - 1
103
104         start = max(0, idx - radius)
105         end = min(len(self.trajectory) - 1, idx + radius)
106
107         window = np.array(self.trajectory[start:end + 1])
108         smooth_x, smooth_y, smooth_a = np.mean(window, axis=0)
109
110         diff_x = smooth_x - self.cumulative_x
111         diff_y = smooth_y - self.cumulative_y
112         diff_a = smooth_a - self.cumulative_a
113
114         smooth_delta = (smooth_x, smooth_y)
115
116         corrected_x = dx + diff_x
117         corrected_y = dy + diff_y
118         corrected_a = da + diff_a
119
120         cos_a = np.cos(corrected_a)
121         sin_a = np.sin(corrected_a)
122
123         M = np.float32([
124             [cos_a, -sin_a, corrected_x],
125             [sin_a, cos_a, corrected_y]
126         ])
127
128         return M, raw_delta, smooth_delta

```

When does the Stabilizer works well and when not?

When the stabilizer works well

- Moderate hand-held camera shake
- Good lighting and minimal blur

When the stabilizer fails

- Motion blur or large frame-to-frame displacements
- Low-texture environments (Not enough features → motion estimation becomes unstable.)
- Large or rapid rotations (only translation + small rotation)